Shortest Path Algorithm with Heaps

Author: Withheld for peer review

Group number: Withheld

Date: 2026-10-16

Abstract

This project compares two min-priority queues for point-to-point Dijkstra searches: a lazy binary heap and a Fibonacci heap with decrease-key. The implementations share one directed-graph representation and differ only in queue management. We evaluate three distance-weighted USA road networks from the 9th DIMACS Implementation Challenge. Each network receives 1,000 reproducible source-target pairs, and both algorithms answer exactly the same pairs. All 3,000 paired distances agree; boundary, randomized, and sanitizer checks also pass. On this Apple M4 system, mean binary-heap query time is 7.393-12.403 ms, whereas mean Fibonacci-heap time is 48.383-78.303 ms. The binary heap is 6.22-6.54 times faster by the ratio of mean times. We explain the algorithmic bounds, experimental procedure, measured variation, and limits of this conclusion. Official dataset download links are supplied without including the large data files.

Contents

1 Introduction **3**

1.1 Background and Motivation 3

1.2 Objectives and Scope 3

2 Data Structure / Algorithm Specification **3**

2.1 Input Graph and Distance Representation 3

2.2 Common Dijkstra Procedure 4

2.3 Lazy Binary Heap 4

2.4 Fibonacci Heap 5

2.5 Correctness Argument 5

3 Testing Results **6**

3.1 Official Inputs and Query Set 6

3.2 Implementation and Timing Environment 7

3.3 Correctness Checks 7

3.4 Runtime Table and Figure 8

3.5 Variation and Paired Comparison 9

3.6 Analysis, Limitations, and Improvements 10

4 Conclusion **10**

5 References **10**

6 Author List **10**

1 Introduction

1.1 Background and Motivation

A weighted shortest-path query asks for a route whose arc weights have the minimum possible sum. Road-network queries are a useful setting because graphs are large, sparse, and naturally represented by outgoing arcs. Dijkstra's algorithm is applicable when all arc weights are nonnegative. It maintains tentative distances, repeatedly selects the smallest one, and makes that vertex's distance final [1]. The choice of min-priority queue changes the cost of these selections and of distance updates.

A binary heap is compact and simple, but its queue operations take logarithmic time. A Fibonacci heap has amortized constant-time insertion and decrease-key, with logarithmic amortized extract-min [2]. Those bounds suggest a theoretical benefit when a search performs many relaxations. Actual elapsed time can differ because queue operations have different constant costs and memory-access patterns. This experiment asks whether the Fibonacci heap's asymptotic advantage translates into lower query time on the selected road networks.

1.2 Objectives and Scope

We implement two complete point-to-point Dijkstra solvers, test their answers, and compare their per-query running times. The binary solver uses the standard priority queue with duplicate entries; the Fibonacci solver holds one node per discovered vertex and uses decrease-key. Both use the same graph, query pairs, distance type, relaxation rule, and early-stop condition. We test the NY, BAY, and COL distance graphs with 1,000 pairs each, exceeding the assignment's minimum of 1,000 pairs. The report includes the requested input links, runtime table, runtime-versus-input-size plot, and interpretation.

The measured task is a single source-target shortest-path query, not all-pairs preprocessing or route reconstruction. Graph loading and pair generation are outside each timed interval. The implementation returns a distance, or `INF` if the target cannot be reached. This distinction matters when comparing results: an apparent speed advantage would be meaningless if one implementation answered a different problem.

2 Data Structure / Algorithm Specification

2.1 Input Graph and Distance Representation

The DIMACS shortest-path format uses a `p sp n m` header and `a u v w` records for directed arcs [3]. The parser ignores comments, allocates one adjacency list per vertex, and stores each arc with its destination and nonnegative integer weight. It rejects an invalid header, an out-of-range endpoint, a negative weight, an unknown record, or an arc count different from the header. Input

IDs begin at 1; internal array indices begin at 0; CSV output converts IDs back to 1-based form.

Each query creates a 64-bit distance array initialized to a large sentinel, except for the source, which is zero. Using 64-bit distances prevents ordinary sums of 32-bit arc weights from overflowing the distance type. The implementation does not claim to handle a valid path whose sum reaches its chosen sentinel (`INT64_MAX/4`); the benchmark paths are far below it. Adjacency lists contain $O(n + m)$ graph data, where n is the number of vertices and m is the number of arcs. The queue and query arrays add space as described below.

2.2 Common Dijkstra Procedure

At any point, `dist[v]` is the best source-to-`v` path length found so far. On removing the smallest valid queue item `(d,u)`, the algorithm inspects each outgoing arc `(u,v,w)`. If `d+w < dist[v]`, it updates `dist[v]` and the queue. A valid extraction of the target ends the query. If the queue becomes empty first, the target is unreachable. The following pseudocode shows the shared behavior; `update` is implemented differently by the two heaps.

```
dist[source] = 0; push(source, 0)
while queue is not empty:
    (d, u) = extract_min()
    if entry is stale: continue       // binary heap only
    if u == target: return d
    for each outgoing arc (u, v, w):
        if v is unsettled and d + w < dist[v]:
            dist[v] = d + w
            update(v, dist[v])
return INF
```

The early stop is important for point-to-point evaluation: a short query need not settle every reachable vertex. Both versions stop at the same logical event, extraction of the target with its final distance. They do not use a heuristic, bidirectional search, or road-specific preprocessing.

2.3 Lazy Binary Heap

The baseline stores `(distance, vertex)` pairs in the C++ binary `priority_queue`. When a better path is found, it pushes a new pair instead of locating and decreasing an older pair. Consequently, a vertex can have several queue entries. At extraction, the code compares the popped key with `dist[vertex]` and discards the pair if the values differ. Since a better key was already inserted, this discarded entry cannot affect the answer. The heap is an array-backed container, and the implementation relies on the standard ordering of pairs.

There are at most $O(m)$ successful relaxations and therefore at most $O(m)$ pushed entries. Each push and pop costs $O(\log m)$ in the worst case. Scanning arcs costs $O(m)$, giving a conservative per-query time bound of $O((n + m) \log m)$ and up to $O(n + m)$ query space [1]. On these sparse

road graphs, m is only a few times n, but the bound alone does not describe the number of vertices settled before an individual target is reached.

2.4 Fibonacci Heap

The Fibonacci version stores one heap node for each discovered, unsettled vertex. Each node contains its tentative-distance key, vertex ID, degree, mark bit, parent, child, and sibling links. Root and child lists are circular doubly linked rings. The heap keeps a pointer to the minimum root. A first discovery inserts the vertex as a root; a later improvement decreases the same node's key. A settled vertex is never reinserted, as permitted by nonnegative arc weights.

Extract-min removes the minimum root, moves its children to the root list, and consolidates roots until no two roots have the same degree. Consolidation links the larger-key root below the smaller-key root. Decrease-key cuts a node when its new key violates its parent's order. A first lost child marks the parent; a further child loss cuts that parent and may cascade upward [2]. These rules maintain heap order and the degree properties used in the amortized analysis. The implementation's key choices are direct node indexing by vertex ID and allocating the node array once per query; it does not allocate a new object on each relaxation.

Standard Fibonacci-heap analysis gives amortized $O(1)$ insertion and decrease-key and $O(\log n)$ extract-min [2]. At most n vertices can be extracted, and at most m arcs can be relaxed, so the resulting Dijkstra bound is $O(m + n \log n)$ per query. The heap has at most n active nodes. Together with graph storage and arrays, total space is $O(n + m)$. These are amortized operation bounds, not a guarantee that a particular short query will run faster than the binary version.

2.5 Correctness Argument

The initial distance of the source is zero and all other distances are infinity. Every finite tentative distance is the length of an actual discovered path, because it is created by extending a previously discovered path over one arc. Relaxation only lowers a tentative distance. The queue therefore always has an entry for the best known distance of each discovered, unsettled vertex: the binary heap may also hold obsolete copies, while the Fibonacci heap changes the one existing node.

Assume the algorithm extracts a valid minimum vertex `u` with distance `d`. If there were a shorter path to `u`, consider the first vertex on that path not yet settled. Its predecessor would already have been settled and would have relaxed the connecting arc. Because weights are nonnegative, that first unsettled vertex would have a tentative distance at most the purported shorter path length, less than `d`. It would have been selected before `u`, a contradiction. Thus `d` is final [1]. Induction over valid extractions proves the distance result, and the same argument justifies stopping at the target. For an unreachable target, the queue eventually empties without extracting it.

This proof assumes that each heap operation preserves the stated minimum and update behavior. The random cross-checks and sanitizers test the implementation of those operations, but finite tests cannot prove every pointer configuration correct. The two solvers are independent in queue

management and share the graph parser and relaxation semantics.

3 Testing Results

3.1 Official Inputs and Query Set

The following files are the official distance-weighted USA road instances used here [3]. Only download links are listed; no graph file is uploaded with this report or its submission packages.

Dataset	Direct download link
NY	USA-road-d.NY.gr.gz
BAY	USA-road-d.BAY.gr.gz
COL	USA-road-d.COL.gr.gz

The header counts and average directed arcs per vertex are shown below. The three inputs grow in vertex and arc count, but they also represent different regions. They are not scaled copies of one graph.

Graph	Vertices n	Arcs m	m/n
NY	264,346	733,846	2.776
BAY	321,270	800,172	2.491
COL	435,666	1,057,066	2.426

For each graph, a `mt19937` generator seeded with `20261016` draws 1,000 source-target pairs uniformly from the valid vertex IDs. The generated vector is retained and passed unchanged to both algorithms. Sampling permits repeated vertices and pairs; no pairs were filtered based on distance. The CSV records each pair and its answer, making the exact benchmark workload reproducible with the same executable and seed. Across the sampled pairs, all targets were reachable. The median returned distances were 468,366.5, 1,053,037.0, and 2,624,367.0 for NY, BAY, and COL, respectively; these are weight units from the distance files, not elapsed times.

3.2 Implementation and Timing Environment

Item	Configuration
Experiment date	2026-10-16
CPU	Apple M4
Operating system	macOS 26.3.1
Compiler and language	Apple clang 21.0.0; C++17
Optimized flags	`-O2 -Wall -Wextra -Wpedantic`
Timer	`std::chrono::steady_clock`, wall elapsed time
Pair generation	`mt19937`, seed `20261016`
Timed repetitions	1 per pair and method; 1,000 pairs per graph
Execution order	All binary queries, then all Fibonacci queries, per graph

A timer starts immediately before a Dijkstra call and stops immediately after it returns. The recorded interval includes allocating and initializing per-query arrays and heap nodes, queue work, arc scans, and termination. It excludes file decompression, graph parsing, query generation, CSV writing, plotting, and report generation. This is therefore an end-to-end query time for this implementation, not the isolated cost of heap operations. Each query is timed once; the 1,000 distinct pairs provide a distribution of workloads rather than repeated timings of one identical query.

The source can be built with `make` from the experiment directory. `make self-test` runs functional checks; `make sanitizer` compiles and runs the same checks with address and undefined-behavior sanitizers. After downloading and decompressing each graph, the command `./build/benchmark data/USA-road-d.NY.gr 1000 20261016 > data/results-NY.csv` illustrates a benchmark run; replace `NY` by `BAY` or `COL` for the others. `python3 scripts/analyze.py` verifies CSV pairing and creates the summary and plot. The local `data/` and `build/` directories are ignored by Git. The submitted PDF contains aggregated results, and the download links permit independent regeneration without uploading the large inputs.

3.3 Correctness Checks

The hand-built graph exercises a seven-weight shortest path that improves an earlier ten-weight candidate, a zero-weight final arc, source equal to target, and an unreachable target. A separate path with two arcs of weight 2,000,000,000 checks that the returned distance is 4,000,000,000 rather than a 32-bit overflow. For randomized checking, 100 directed graphs with 30 vertices and 200 randomly weighted arcs each contribute 20 random source-target pairs. Both algorithms returned equal answers for all 2,000 small-graph pairs. AddressSanitizer and UndefinedBehaviorSanitizer

reported no error during this self-test.

During official benchmarking, the program computes all binary answers, then checks each Fibonacci answer against the corresponding binary result. It exits with an error on the first mismatch. The three benchmark programs completed successfully, so all 3,000 paired distances matched. This equality is an empirical check, not an independent oracle: a shared parser or relaxation bug could affect both methods. The small hand-built cases address some such shared errors, while a formal proof and broader tests would be needed for stronger assurance.

3.4 Runtime Table and Figure

The mean and median below are calculated from the 1,000 per-query intervals for each graph and method. Total time is the sum of those intervals, not an independently timed batch. Values are rounded to three decimals after aggregation.

Graph	Vertices	Arcs	Heap	Mean (ms)	Median (ms)	Total (s)
NY	264,346	733,846	Binary	7.393	7.286	7.393
NY	264,346	733,846	Fibonacci	48.383	47.353	48.383
BAY	321,270	800,172	Binary	9.425	8.699	9.425
BAY	321,270	800,172	Fibonacci	58.620	57.833	58.620
COL	435,666	1,057,066	Binary	12.403	12.317	12.403
COL	435,666	1,057,066	Fibonacci	78.303	76.375	78.303

The plot places the three graph vertex counts on the horizontal axis and mean milliseconds per query on the vertical axis. Both series rise from NY to COL, while the Fibonacci series stays well above the binary series. Each point summarizes 1,000 queries; the lines connect measured points and do not imply measurements at intermediate graph sizes. The two methods share one vertical scale so their absolute costs can be compared directly.

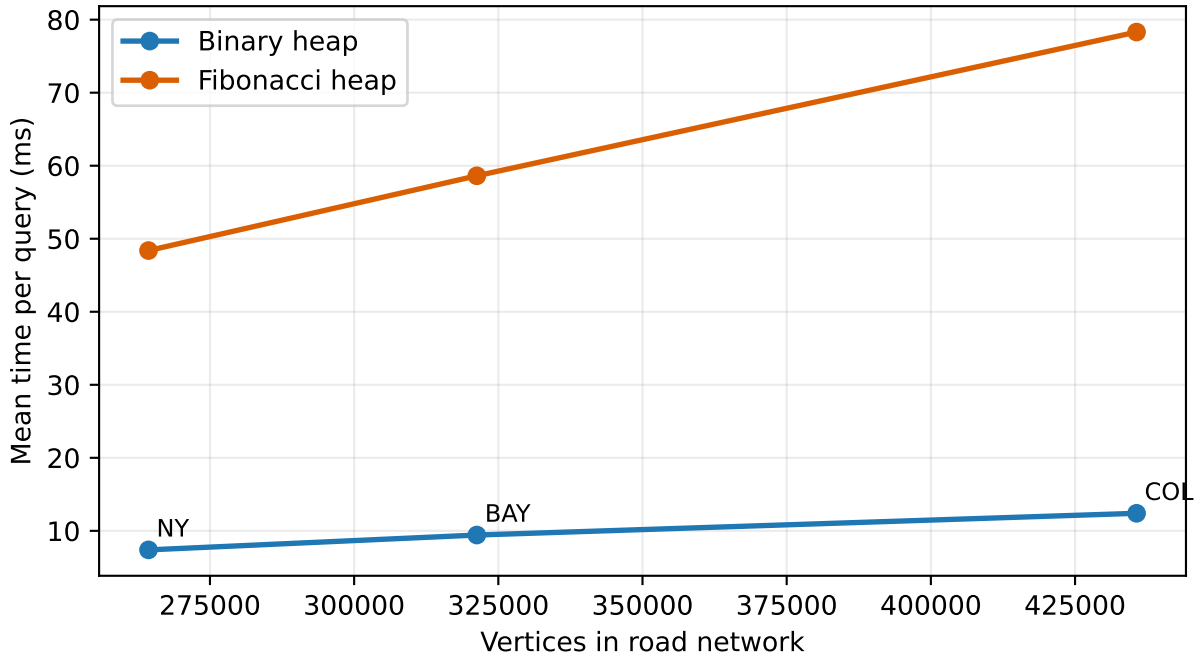


Figure 1: Mean shortest-path query time versus graph size

3.5 Variation and Paired Comparison

Query times vary substantially because source-target pairs cause different search extents. To show variation without hiding it in a mean, the table gives the 25th and 95th percentiles of the 1,000 individual timings. Percentiles use linear interpolation between sorted observations.

Graph	Heap	25th percentile (ms)	95th percentile (ms)
NY	Binary	3.433	14.212
NY	Fibonacci	23.385	92.836
BAY	Binary	4.301	17.367
BAY	Fibonacci	28.636	109.921
COL	Binary	6.174	23.632
COL	Fibonacci	40.150	146.637

The ratio of Fibonacci mean to binary mean is 6.54 for NY, 6.22 for BAY, and 6.31 for COL. This ratio compares two methods on identical pairs for each graph, but it is a ratio of aggregate means, not a claim that every pair has that exact speedup. The binary method also has the lower median and 95th-percentile time in all three networks. No sampled target was unreachable, so these comparisons describe reachable random pairs.

3.6 Analysis, Limitations, and Improvements

The Fibonacci heap's asymptotic advantage depends on the mix of decrease-key and extract-min operations and whether its amortized savings overcome constant costs [2]. The binary heap uses a compact array of pairs; the Fibonacci heap manipulates linked trees. The data are consistent with those mechanisms explaining the gap, but we did not measure operation counts or hardware events, so we cannot assign a measured fraction of time to either mechanism.

First, the runs use one machine and one timing pass per pair. Scheduling, frequency changes, cache state, and other processes can perturb wall-clock timing. Alternating heap order and repeating the full experiment on a quiet machine would give more stable estimates. Second, the three networks differ in geography and average degree as well as size; the rising lines cannot establish a scaling law from size alone. More official road graphs and controlled graph families would help separate those effects. Third, the comparison is between the particular lazy binary implementation and this particular Fibonacci implementation. An indexed binary heap, different memory layout, or optimized Fibonacci heap could change the gap. These limits qualify the claim; they do not change the observed timings and answer agreement on the tested inputs.

4 Conclusion

Both Dijkstra implementations, including the decrease-key Fibonacci heap, agreed on all 3,000 road-network query distances. The binary heap was 6.22-6.54 times faster by the ratio of mean times, making it the practical choice for these inputs and this code. Fibonacci heaps retain the stronger amortized bound [2]. Repeated, order-balanced runs would strengthen the performance conclusion.

5 References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022, ch. 22. Publisher.

[2] M. L. Fredman and R. E. Tarjan, "Fibonacci heaps and their uses in improved network optimization algorithms," *Journal of the ACM*, vol. 34, no. 3, pp. 596-615, 1987, doi: 10.1145/28869.28874.

[3] 9th DIMACS Implementation Challenge, "Shortest Paths: Benchmark Instances and File Formats." Download page; format specification.

6 Author List

Author information withheld for peer review.